(Special Session)

Beyond Verilog: Agents for Emerging HDLs

Farzaneh Rabiei
University of California Santa Cruz
Computer Science and Engineering
Santa Cruz, USA
frabieik@ucsc.edu

Mark Zakharov
University of California Santa Cruz
Computer Science and Engineering
Santa Cruz, USA
mzakharo@ucsc.edu

Jose Renau
University of California Santa Cruz
Computer Science and Engineering
Santa Cruz, USA
renau@ucsc.edu

***Abstract*—Large Language Models (LLMs) are transforming the programming language landscape, but their effectiveness is limited for niche Hardware Description Languages (HDLs) due to insufficient training data. This paper introduces HDLAgent, an agentic AI framework that enhances LLMs' proficiency with underrepresented HDLs.**

Our agent employs two key mechanisms: HDL description summaries that facilitate knowledge transfer from familiar languages like Verilog, and enhanced grounding through compiler error feedback. We demonstrate that HDLAgent significantly improves code generation across multiple HDLs. For example, PyRTL's functional correctness rate improves from 0% to 35% with Mixtral 8x7B, and Chisel's success rate increases from 0% to 59% with GPT-3.5-turbo. For small code samples (under 25 lines), HDLAgent achieves over 90% success across all tested HDLs. Beyond performance metrics, we provide insights on scalability limitations with increasing code complexity and propose modifications to HDL designs that could better accommodate LLM-based generation.

***Index Terms*—Large Language Models, Hardware Description Languages, AI-enhanced coding, Code generation, Verilog, PyRTL, Chisel, DSLX.**

I. Introduction

Large Language Models (LLMs) are transforming programming through intelligent code generation and context-aware suggestions. However, LLMs currently offer limited support for niche Hardware Description Languages (HDLs) such as Chisel3 [1], PyRTL [2], and DSLX [3]. This limitation threatens to impede the adoption of innovative HDLs critical for advancing hardware design methodologies.

Given that most high-performance LLMs are closed-source, directly retraining them for specialized HDLs is impractical. Instead, inference-time agentic approaches that enhance LLM capabilities without requiring lengthy training cycles are essential to prevent LLMs from becoming barriers rather than enablers in hardware development.

An alternative to training is to use Agents. We introduce **HDLAgent**, an agentic AI system designed to enhance LLM performance for underrepresented HDLs. HDLAgent employs iterative feedback loops and domain-specific knowledge augmentation to bridge the gap between well-supported languages like Verilog and newer HDLs. Specifically, HDLAgent introduces:

- **HDL Description Summaries:** Concise summaries of HDL semantics that facilitate knowledge transfer from familiar languages to new ones, optimized for LLM reasoning.
- **Enhanced Grounding Messages:** Compiler error feedback loops that iteratively refine code generation by providing targeted correction examples.

Our evaluations tests functional correctness across four HDLs. Results demonstrate that HDLAgent significantly improves code generation success rates—defined as producing functionally correct code that passes compilation and logical equivalence checks. For example, using HDLAgent with Mix-8x7B increases Chisel success rates from 3% to 44%, while GPT-3.5o's DSLX success rate improves from 3% to 48%. For small examples (under 25 lines), HDLAgent achieves over 90% success across all tested HDLs.

HDLAgent shows that it is possible to create new HDLs and leverage LLMs with Agentic flows without requiring to retrain the LLMs. The key contributions of this paper are:

- **HDLAgent Framework:** An inference-time agentic system that enhances LLM performance for niche HDLs without requiring model retraining.
- **Performance Analysis:** Detailed evaluation showing HDLAgent's effectiveness across different HDLs, LLMs, and problem complexities, revealing both capabilities and limitations.
- **HDL Design Guidelines:** Strategic recommendations for designing future HDLs that better accommodate LLM-based generation, informed by error pattern analysis.

II. Related Work

Improving LLM performance typically involves fine-tuning or agent-based techniques. HDLAgent avoids fine-tuning because it is not an option for closed-source LLMs and new HDLs have limited data.

Agents [4] iterate through LLMs with three main techniques to improve performance: Self-reflection, few-shot or RAG, and grounding.

Self-reflection techniques involve LLMs refining their outputs through iterative processes, as in Chain-of-Thought (CoT) [5]. This is a rapidly evolving field, with recent works [6] proposing optimization methods to find the best prompts. Few-shot in-context learning [7], [8] and RAG [9] employ instructions and several examples related to the question. Grounding uses external tools like compilers for validation and correction [10], and recent research has extended these agent-

979-8-3315-2037-3/25/$31.00 © 2025 IEEE

based methods to address coding challenges in architecture-specific programming languages [11].

These agentic ideas have also been applied to Verilog. Most notably, AutoChip [12] uses testbench feedback to ground the generated Verilog. RTLFixer [13] uses ReAct [14] for self-reflection, and RAG for grounding compiler errors. HDLDebugger [15] fine-tunes CodeLlama to fix the code generation. VerilogCoder [16] improves results by enhancing the testbench feedback.

Each has different design techniques, but HDLAgent distinguishes itself by supporting multiple new HDLs.

III. HDLAgent

HDLAgent is an AI Agent (refer to Section II) designed to adapt advanced AI coding techniques to Hardware Description Languages (HDLs) not typically included in LLM training data.

HDLAgent leverages LLMs' transfer learning abilities [17], [18], enabling them to handle underrepresented HDLs by transferring knowledge from well-known languages like Verilog to new HDLs such as PyRTL. This process mirrors human learning mechanisms [19].

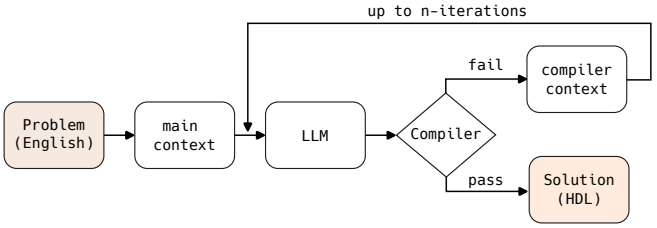

Fig. 1: HDLAgent flow leveraging compiler feedback.

As illustrated in Figure 1, HDLAgent addresses the critical challenge of limited HDL-specific knowledge. It employs two primary memory components to bridge this gap: the "main context" and the "compiler context". The "main context" (described in Section III-A) offers a succinct summary of HDLs along with targeted examples. The "compiler context" (Section III-B) enhances code generation by integrating compiler feedback, grounding the output within practical and executable constraints.

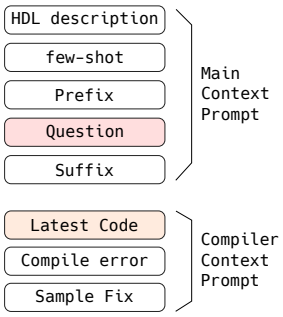

Fig. 2: HDLAgent Main and Compiler context prompt components.

A. Main Context

The main context informs the LLM about the specific HDL, consisting of an HDL description, few-shot examples, prefix, and suffix. Figure 2 illustrates this main context, which comprises four key elements: HDL description, few-shot examples, Prefix, and Suffix.

The HDL description provides a concise summary tailored to the LLM's familiarity with the HDL. We use the description that works best with Mix-8x7B and GPT-3.5n due to their lower proficiency in certain HDLs. Few-shot examples are crucial to help LLMs avoid common pitfalls, especially in HDLs with unique syntaxes.

Prefix and suffix guide the LLM, with the prefix introducing the problem and the suffix providing instructions to ensure output adheres to HDL syntax without extraneous explanations.

Interestingly, even for LLMs capable of processing entire HDL reference manuals within their context window, utilizing a summary enhances both success rate and token efficiency. Our evaluation shows an improvement in success rates when employing an HDL description. The intuition is that focusing on the Verilog differences is more important than providing a lengthy description of the language.

Since including a complete tutorial is neither practical nor advantageous for the evaluated LLMs, we use an HDL description summary instead. To generate these summaries, we leverage LLMs with large context windows, specifically GPT-4 and GPro-1.0, to condense the HDL reference manuals.

For PyRTL and Chisel, our evaluation revealed that the most effective prompt was generated by GPT-4 using the following instruction: "PyRTL is a Hardware Description Language with the following reference documentation and tutorial. Create documentation useful for LLMs trying to generate PyRTL code. The generated documentation should include code snippets and highlight any language syntax that is atypical for HDLs."

For DSLX, the optimal summary was produced by GPro-1.0 using a similar prompt, with the addition of "Be concise and avoid examples with similar syntax." at the end. This minor variation in the prompt yielded the best results.

Complementing the HDL description, the "main context" provides few-shot examples to illustrate common HDL operations and potential areas of confusion. These examples cover bit operations, reductions, loops, multiplexing, and a multiply-add block. While the HDL Description can include some examples, it is important to cover these basic operations with simple examples, as LLMs tend to revert to incorrect syntax.

The bit operations example demonstrates simple bit manipulation and concatenation, while the reduction example showcases a basic NOR reduction over a given input.

Loops can be particularly confusing in some HDLs. For instance, in DSLX all variables are immutable, but loops have a special syntax for accumulator variables. Including relevant examples in the HDL context helps address cases where the LLM needs to create a loop.

The Prefix follows the few-shot examples, briefly directing the original Question with a statement such as the following

for DSLX: "The following statements describe the problem to be addressed in DSLX."

The Suffix, appended after the question, serves to limit the scope of the problem and provide specific instructions. It includes directives like "respond with valid program syntax only, without additional English explanations" and tailors HDL input and output formats. Handling I/Os is crucial, especially for HDLs with multiple output options, necessitating instructions to maintain output integrity and naming consistency. For DSLX, the Suffix includes directives like "do not split the outputs into individual bits" and "variables assigned to the output struct should have the same name as the struct fields." This Suffix concept is essential even when using Verilog, as it can employ interfaces, structs, or plain Verilog-2001 syntax.

The HDLAgent Suffix facilitates interfacing between different HDLs and Verilog. However, maintaining consistent naming conventions and common syntax remains a critical issue that the Suffix must address, even in single-HDL use cases.

B. Compiler Context

HDLAgent's compiler context employs an iterative approach to rectify inaccurately generated HDL code. This process grounds the LLM-generated code by providing feedback on potential errors or hallucinations. Before submitting the LLM output to the compiler, HDLAgent identifies the code section. This step is crucial, as LLMs may generate English explanations despite explicit instructions to avoid them.

When the generated program fails compilation, producing a compiler error, HDLAgent constructs a query (illustrated in Figure 2). This query begins with the "main context," disregarding any non-code responses. It then incorporates the latest code snippet, followed by a statement indicating "the previous code has the following compile error," succeeded by the specific compiler error message. If HDLAgent possesses an example fix for addressing the compiler error, it appends this "sample fix" to the context.

The sample fix methodology is analogous to RTLFixer [13], which provides explanations for resolving Verilog error messages. HDLAgent extends this concept to cover multiple HDLs, elucidating the special syntax requirements of a given HDL when necessary.

HDLAgent presents the entire latest code snippet in its query. We experimented with a method inspired by CWhy [20], which focuses on a few lines of code surrounding the compiler error message. While this approach worked for some LLMs like GPT-4, it proved less effective with others. Although this delta approach reduces token usage, it led to increased error rates, prompting us to exclude it from our evaluation. As LLMs continue to evolve, this approach may warrant reconsideration in future iterations.

C. Prompt Optimizations

Besides the previous main and compiler context there are also several subtle but important optimizations:

- Placing the prompt after the context achieves better results [21].
- HDLAgent approach avoids the chat-like history with all the previous code generations and fixes. Keeping the original question iteration but not the compiler error fixes achieves better results [12]. We did a quick test with HDLAgent and DSLX. Avoiding a history with all the error fixes had a 5% improvement in GPT-4 and a 27% in Mix-8x7B.
- Most LLMs generate code snippets in quoted sections, but not always. Even worse, it is common to write English explanations even thought the prompt explicitly asks to just write code. To address this, for each language we have a filter/detector that removes English and finds code boundaries. For example in Verilog it allows preprocessor directives and code between module and endmodule. Without this, some smaller LLMs fail very frequently.

IV. Setup

Table I lists all the languages used in the evaluation and the compiler versions used by this paper. When a date is provided, it corresponds to the top-of-tree version at that given month. For Quality of Results (QoR), we use Yosys synthesis results.

TABLE I: Language Tools and Versions

Language	Tool	Version
Verilog	Yosys	0.35
Chisel	FIRRTL	3.5.0-RC2
PyRTL	PyRTL compiler	0.10.2
DSLX	XLS	3/2024

TABLE II: LLMs used in the evaluation

LLM	Version	Date	Context
GPT-4	gpt-4-1106-preview	4/23	128000
GPT-3.5n	gpt-3.5-turbo-0125	9/21	16385
GPT-3.5o	gpt-3.5-turbo-1106	9/21	16385
GPro-1.0	gemini-1.0-pro-001	2/24	32720
Mix-8x7B	Mixtral-8x7B-instruct	12/23	32768
Mix-8x22B	Mixtral-8x22B-v0.1	3/24	32768

Table II shows the LLMs used. Many LLMs, including GPT-3.5o, are non-deterministic, producing varying outcomes for identical prompts. While OpenAI's new API with a seed aims to address this, it is not yet widely implemented. For fair evaluation, we avoid deterministic settings and perform 1, 5, or 10 runs based on the top@k parameter.

V. Evaluation

A. Overall Results

To comprehensively assess HDLAgent's performance across various LLMs, we evaluate each HDL (Chisel, PyRTL, DSLX, and Verilog) against four benchmark tests: VH (VerilogEval-Human), VM (VerilogEval-Machine), HC (HDLEval-Comb), and HP (HDLEval-Pipe).

While VerilogEval tests comprise several Verilog-specific questions, they do not fully demonstrate the potential of the LLM/HDLAgent combination as effectively as HDLEval (HC, HP). This is primarily due to VerilogEval's inclusion

of Verilog-specific instructions in some tests, such as implementing a D latch using an "always" block. Such tests are not suitable for evaluating languages other than Verilog. Consequently, we utilize VH and VM primarily for Verilog or as a reference point, while focusing our evaluation on HDLEval (HC and HP).

The results are broken down into five key components to delineate the incremental benefits provided by HDLAgent:

- *Base*: Represents the baseline performance of an LLM without HDLAgent enhancements but includes basic I/O formatting and general code generation guidelines.
- *Description*: Adds a concise HDL Description to the LLM context, improving specificity (see Section III-A for details).
- *Few-shot*: Adds language-specific few-shot examples.
- *Compile*: Incorporates compiler feedback with up to eight iterations to refine the generated code, optimizing accuracy.
- *Fixes*: Performs the same iterations as *Compile*, but for each iteration, provides a suggestion alongside a generic example on how to address the specific compiler error.

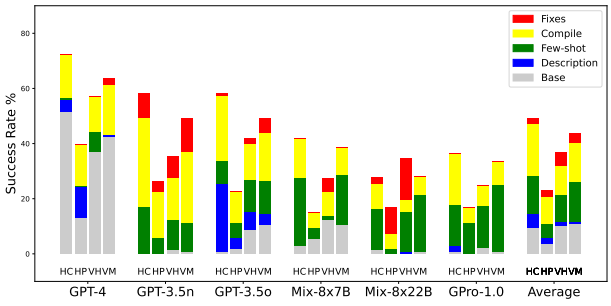

Fig. 3: HDLAgent improves Chisel across all LLMs.

Chisel (Figure 3), a Scala-based HDL, presents a unique challenge and opportunity. Most LLMs are familiar with Scala but have limited knowledge of Chisel. While several LLMs demonstrate familiarity with its basic syntax, only GPT-4 initially performs adequately with Chisel (52% success rate). All other LLMs exhibit a mere 3% success rate or less. Both the "main context" (comprising Description and Few-shot components) and the "compiler context" (including Compile and Fixes elements) provide substantial benefits, underscoring the necessity of all these components. Notably, with HDLAgent, GPT-3.5o and GPT-3.5n outperform even high-performing LLMs like GPT-4 in its baseline state. Furthermore, HDLAgent significantly enhances GPT-4's performance, elevating its success rate to 72%.

Examining the average performance across all LLMs reveals that each component of HDLAgent contributes significantly to the overall improvement. This underscores the comprehensive and synergistic nature of HDLAgent's approach to enhancing LLM performance in Chisel code generation.

PyRTL (Figure 4), a Python-based Domain Specific Language (DSL), presents challenges similar to those of Chisel. OpenAI's LLMs (GPT-4, GPT-3.5n, GPT-3.5o) demonstrate some

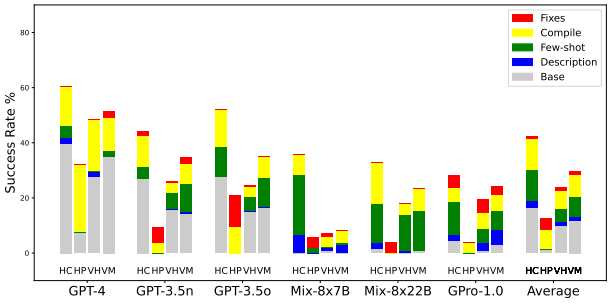

Fig. 4: HDLAgent improves PyRTL across all LLMs.

capability in passing several tests without the assistance of HDLAgent (Base) due to their strong baseline performance in Python; however, their success rates for PyRTL remain low, ranging from 27% to 40%. When the test cases benefit from HDLAgent features, these success rates significantly improve, increasing to a range of 44% to 60%. As observed in the Chisel evaluation, HDLAgent's performance boost stems from multiple factors.

Mirroring the Chisel results, all components of HDLAgent prove important for PyRTL, with the "compiler context" (*Compile* + *Fixes*) playing a particularly crucial role. This heightened importance of compiler iterations for both PyRTL and Chisel can be attributed to their nature as DSLs and the specific error messages they generate. By leveraging PyRTL or Chisel compiler error messages, HDLAgent iterates to refine the code. Since the baseline LLM has some familiarity with the language, it can effectively interpret these error messages and iterate to fix mistakes.

LLMs often confuse the syntax of DSL host languages (Python for PyRTL, Scala for Chisel) with HDL-specific syntaxes. The HDL Description significantly aids compiler iterations in rectifying these mistakes, demonstrating synergy between the Description and Compile passes. Although not shown in the Figure, enabling only the Compile pass without the HDL Description and Few-Shot examples yields substantially lower overall improvement. This issue could be mitigated if PyRTL or Chisel compilers generated errors that clarified the distinction between base languages and DSLs, perhaps by providing a single few-shot example.

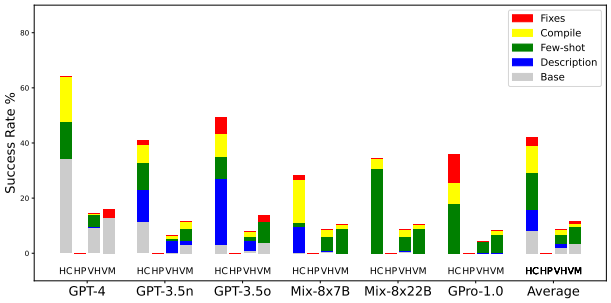

Fig. 5: HDLAgent improves DSLX HDLEval-Comb across all LLMs.

DSLX (Figure 5), a Rust-like language, presents unique challenges for implementing a Rust-like syntax that it is not fully compatible with Rust. Due to its limitations on arbitrary pipelining, DSLX cannot be evaluated against HDLEval-Pipe and performs poorly with VerilogEval. While GPT-4 demonstrates some DSLX knowledge, HDLAgent significantly enhances results across all LLMs.

Unlike Chisel and PyRTL, DSLX is not a DSL. Consequently, the "main context" (*HDL Description + Few-Shot*) emerges as the primary factor in HDLAgent's improvement. Explaining the Rust-like syntax and providing examples proves more crucial than grounding results with compile errors.

This shift in importance from compiler feedback to Description stems from DSLX's unique syntax. While it resembles Rust, not all Rust syntax is valid in DSLX. In contrast, Chisel and PyRTL accept all Scala and Python syntax, respectively. Without clear guidance on DSLX's specific syntax, LLMs struggle to generate correct code.

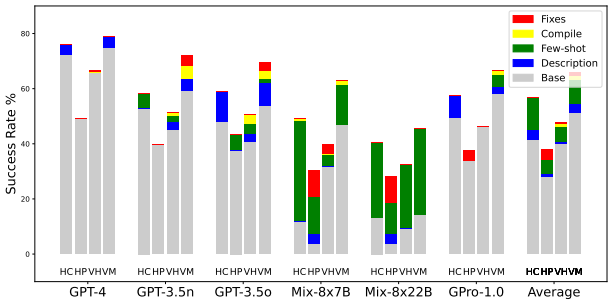

Fig. 6: Verilog succeeds across benchmarks and LLMs .

Verilog (Figure 6) demonstrates the best overall performance in the *Base* condition (without HDLAgent), as expected due to extensive training with Verilog syntax. It is also the only fair case for VerilogEval use. HDLAgent minimally impacts models already proficient in Verilog but significantly enhances Mix-8x7B and Mix-8x22B, which have some Verilog knowledge, illustrating effective knowledge transfer even with limited Verilog familiarity.

Compiler iterations provide little benefit in error recovery for Verilog. This is due to LLMs' higher proficiency in Verilog syntax. Manually analyzing the HC results, only 5 out of 134 tests showed Verilog syntax errors, with just one potentially benefiting from improved error messaging. As a result, using Slang [22] instead of Yosys [23] did not improve results for GPT-3.5o despite producing more descriptive error messages.

Unexpectedly, Mix-8x22B underperforms Mix-8x7B, likely due to difficulty following directions. An "instruct" model might perform better, but the non-instruct model was used to assess HDLAgent's impact across LLMs.

HDLAgent successfully enables LLMs to use new HDLs. Comparing GPT-3.5o and GPT-3.5n across HDLs shows consistent relative performance regardless of the LLM used. For example, with GPT-4, Verilog achieves a 76% success rate, while PyRTL, the lowest, reaches 60%. This pattern holds across all tested LLMs. Even the worst-performing LLM (Mix-8x22B) achieves a 53% success rate with Verilog and 28% with PyRTL, a significant improvement from the zero success rates many LLMs had without HDLAgent.

B. Context Insights

This section offers insights into the selection of HDL Description and few-shot context. One straightforward approach is to utilize the full reference manual directly for the specific language. While this is feasible for models with large context windows such as GPT-4, Mix-8x7B, and GPro-1.0, it generally proves less effective than employing a summarized HDL description. For instance, using a full reference instead of a summary yields no change in results for GPro-1.0, but reduces the success rate from 77% to 66% for GPT-4, and from 59% to 33% for Mix-8x7B.

Interestingly, adding few-shot always improves results, and removing HDL Description and just keeping few-shot examples is a reasonable alternative. In some HDL/LLM combinations like Chisel/GPT-3.5n, using either Few-shot or Description works. For other combinations like DSLX/Mix-8x7B, HDL Description helps but few-shot is necessary. Optimal results require both Few-shot and HDL Description.

C. QoR Insights

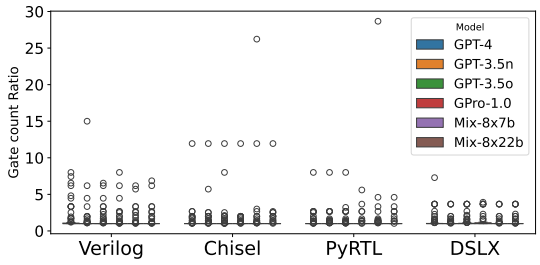

Fig. 7: QoR is consistent across LLMs but different across HDLs.

Current tests HDLs are relatively small (under 500 LoC of equivalent Verilog), with many being purely combinational. Consequently, frequency or power QoR metrics are not relevant. Instead, we approximate QoR as the ratio of gates used compared to the best known implementation for each module request.

Figure 7 illustrates the gate count ratio relative to the optimal implementation. A ratio of 1 indicates optimal gate count, while 2 signifies double the optimal count. For many designs, we used the generated code as the optimal result.

The plot reveals significant QoR variation compared to the best implementations. Typically, averages are skewed by one or two outliers. For example, in PyRTL generated by Mix-8x7B, the average gate count ratio is 1.63, but drops to 1.12 when two outliers are removed. This suggests that LLMs occasionally generate highly inefficient code, but such instances are infrequent.

A second observation indicates that GPT-4 may appear to underperform; however, this is partly due to its ability to

successfully implement larger and more complex designs that are difficult to optimize, which affects the overall results. A third observation is that the efficiency of code generated by various LLMs is generally comparable. Among these, DSLX appears to be the most efficient, albeit by a slim margin. In DSLX generated by GPT-4, 80% of the produced code achieves the optimal 1:1 ratio. This suggests that an efficient compiler like XLS, combined with a popular syntax, can yield superior results for generated HDL code.

D. Problem Size Insights

HDLEval-Comb comprises 134 tests, with some being relatively small, containing only a few lines of code, while others are significantly more extensive. The best proxy for complexity is not the input problem itself, but the lines of code (LoC) required to implement such a problem in a specific language (Verilog in this case).

Figure 8 illustrates the success rate for HDLEval-Comb using GPT-4 across four different problem sizes: under 25 LoC, 25-50 LoC, 50-75 LoC, and over 75 LoC of equivalent Verilog. A clear degradation in performance is evident as the required code size increases.

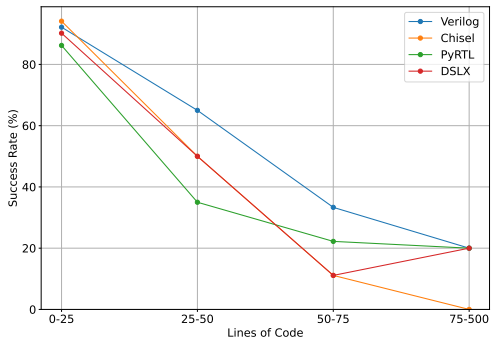

Fig. 8: Even with best LLM (GPT-4), performance degrades as Lines of Code for generated output increases.

As demonstrated in Section V-A, GPT-4 is the best-performing LLM with HDLAgent, achieving average success rates of 72% for Chisel, 60% for PyRTL, 64% for DSLX, and 76% for Verilog. Figure 8 reveals that for small problems, HDLAgent performs consistently across all HDLs, with over 90% success rate. Conversely, for large problems exceeding 75 LoC, all HDLs, excluding Chisel, have a consistently low 20% success rate. The performance difference between HDLAgent and Verilog is most pronounced in medium-sized problems ranging from 25 to 75 LoC.

Another observation is that this degradation happens with different degrees across all LLMs. Figure 9 shows Verilog success rates across different LLMs. While curves vary slightly, the overall trend is consistent: performance degrades as the output problem size increases. Notably, Mixtral models (Mix-8x7B and Mix-8x22B) show some improvement for larger problems, which may reflect random variation but suggests these models handle larger problems more resiliently.

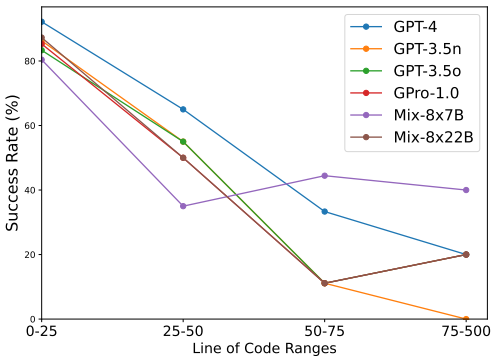

Fig. 9: HDLEval-Comb performance degrades for larger Verilog codes across LLMs.

These results highlight two key points: First, solving larger problems remains challenging, with Mix-8x7B achieving success rates below 40% even with HDLAgent. Second, HDLAgent ensures strong performance for small examples across HDLs, underscoring its utility for new HDL learners generating small code snippets.

VI. Future Work and Conclusions

Developing a new HDL is a difficult task. This work addresses one of the potential challenges, namely how to work with LLMs when there is no training because it is a new HDL.

This paper HDLAgent enhances the ability of LLMs to generate code for HDLs not represented in training datasets. The evaluation considers multiple HDLs with different degree of LLM support such as Chisel, PyRTL, and DSLX. Although the HDLs have limited knowledge on the HDLs, HDLAgent removed most of the syntax errors leaving mostly semantics errors.

The evaluation show that HDLAgent achieves a success rate of over 90% on concise examples across all HDLs, making it an excellent tool for educational purposes in teaching new HDL languages. However, the performance of HDLAgent and traditional LLM approaches tends to decline with larger or more complex designs. For instance, even advanced LLMs like GPT-4 see a drop in success rates for Verilog projects exceeding 75 lines of code.

HDLAgent is open-source and enables further research to address currently identified challanges like the QoR insights (Section V-C) and problem size (Section V-D).

While HDLAgent significantly improves performance—raising Verilog success rates of GPT-4 from 34% to 72%—scalability challenges remain for more intricate designs.

Acknowledgments

We like to thank the reviewers for their feedback on the paper. This work was partially funded by Google Academic Research Award.

REFERENCES

[1] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, "Chisel: constructing hardware in a scala embedded language," in *DAC Design Automation Conference 2012*. IEEE, 2012, pp. 1212–1221.

[2] J. Clow, G. Tzimpragos, D. Dangwal, S. Guo, J. McMahan, and T. Sherwood, "A pythonic approach for rapid hardware prototyping and instrumentation," in *Field Programmable Logic and Applications (FPL), 2017 27th International Conference on*. IEEE, 2017, pp. 1–7.

[3] Google, "XLS Website," https://github.com/google/xls/, 2022.

[4] W. Zhou, Y. E. Jiang, L. Li, J. Wu, T. Wang, S. Qiu, J. Zhang, J. Chen, R. Wu, S. Wang, S. Zhu, J. Chen, W. Zhang, N. Zhang, H. Chen, P. Cui, and M. Sachan, "Agents: An open-source framework for autonomous language agents," 2023.

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023.

[6] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," 2023.

[7] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. A. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 1950–1965, 2022.

[8] T. Ahmed and P. Devanbu, "Few-shot training llms for project-specific code-summarization," *arXiv preprint arXiv:2207.04237*, 2022.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021.

[10] H. Wang, Z. Liu, S. Wang, G. Cui, N. Ding, Z. Liu, and G. Yu, "Intervenor: Prompt the coding ability of large language models with the interactive chain of repairing," 2023.

[11] W. Liu, H. Wang, J. Li, K. Zhang, R. Chen, and M. Zhou, "Adaptive self-improvement llm agentic system for ml library development," 2025.

[12] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "Autochip: Automating hdl generation using llm feedback," 2023.

[13] Y.-D. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language models," 2024.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," 2023.

[15] X. Yao, H. Li, T. H. Chan, W. Xiao, M. Yuan, Y. Huang, L. Chen, and B. Yu, "Hdldebugger: Streamlining hdl debugging with large language models," 2024.

[16] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool," 2024. [Online]. Available: https://arxiv.org/abs/2408.08927

[17] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[18] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," *arXiv preprint arXiv:2308.10144*, 2023.

[19] J. C. Scholtz, *A study of transfer of skill between programming languages*. The University of Nebraska-Lincoln, 1989.

[20] (2023, November) CWhy. website. [Online]. Available: https://github.com/plasma-umass/cwhy

[21] P. Islam, A. Kannappan, D. Kiela, R. Qian, N. Scherrer, and B. Vidgen, "Financebench: A new benchmark for financial question answering," 2023.

[22] "slang - SystemVerilog Language Services," https://github.com/MikePopoloski/slang, online; accessed on 5 August 2021.

[23] C. Wolf, "Yosys Open SYnthesis Suite," https://github.com/YosysHQ/yosys, 2022, Online; accessed on December 2022.